

National Institute of Technology Karnataka Surathkal
Department of Information Technology



IT 252 DATABASE SYSTEMS

Database System Concepts and Architecture

Dr. Jayashree T R

Course Outline

Course Plan: Theory:

Part A: Parallel Computer Architectures

Week 1: Introduction to the course, highlighting the data and databases, and related basic concepts. Advantages and need for a database system.

Week 2: Attributes, tuples, relational schema, conceptual model, introduction to SQL-DML, DDL, creating relations/ tables, access and manipulate data.

Week 3,4: Data model, relational model, and SQL, relational data model and relational database constraints, SQL data definition and data types, specifying constraints, basic retrieval queries, complex queries, triggers, views, schema modification.

Week 5-6: ER model development, entity types/sets, attribute, relationship types/sets, simple employee database conceptual design using ER concepts, ER to relational mapping algorithm, EER model.

Course Outline

- Week 7,8 : Functional dependency definition, need of normalization, anomalies and redundant information in tuples, normalization steps (1NF, 2NF, 3NF), BCNF, introduction to the higher normal forms.
- Week 9: Procedural query languages: relational algebra and SQL-SELECT, PROJECT, RENAME, binary and JOIN operations with examples.
- Non-Procedural query languages: relational calculus, and SQL-tuple relational calculus, existential and universal quantifier with examples.
- Week 10,11 : Transaction management, schedule and serializability, concurrency control, 2-phase lock, recovery mechanism: undo/redo values.
- Week 12,13: Disk storage- basic file structures, ordered/unordered. binary search, hashing, indexing, importance of indexes, primary and secondary indexing methods, clustered, B tree, B++ trees.
- Week 14: Current trends in database system, introduction to data warehousing, data mining.
- Practical: MySQL commands
- Project: Team of 2 or 3 members

Basic client/server DBMS architecture

- Client module:
 - runs on a mobile device, user workstation, or personal computer (PC)
 - handles user interaction and provides GUIs (graphical user interfaces) for PCs
- Server module
 - handles data storage, access, search, and other functions.

Data Models, Schemas, and Instances

Data abstraction

- Suppression of details of data organization and storage.
- Highlighting of the essential features for an improved understanding of data.

Data model

- Collection of concepts that describe the **structure of a database** (means the **data types, relationships, and constraints** that apply to the data).
- Provides means to achieve data abstraction
- Also includes a set of **Basic Operations** to specify retrievals and updates on the database.
- Dynamic aspect or behavior of a database application: Allows the database designer to specify a set of **valid operations** allowed on database objects.

Categories of Data Models:

Broad Categories:

- **High-level or conceptual data models**
 - Close to the way many users perceive data.
 - Is a high-level, abstract representation of the data and its relationships within a system.
 - It is used primarily in the early stages of database design to show the essential business requirements without getting into technical details.

Categories of Data Models:

Broad Categories:

- **Low-level or physical data models**
 - Describe the details of how data is stored on computer storage media
 - generally meant for computer specialists, not for end users.
- **Representational data models**
 - Easily understood by end users
 - Also similar to how data organized in computer storage
- **Object data model**
 - is a data model that represents data in terms of **objects**, their attributes, and their behaviors (methods).
 - Closer to conceptual data models

Categories of Data Models:

Broad Categories:

- **Network data models**

- Data is represented as a network or graph with nodes and edges.
- Data is linked via pointers, with one record being the owner and others being members.

- **Hierarchical data models**

- Organizes data in a tree-like structure where records (nodes) are connected via parent-child relationships.

- **Relational data model**

- Used most frequently in traditional commercial DBMSs
- Is a way to structure data using tables (called **relations**) to represent entities and their relationships.

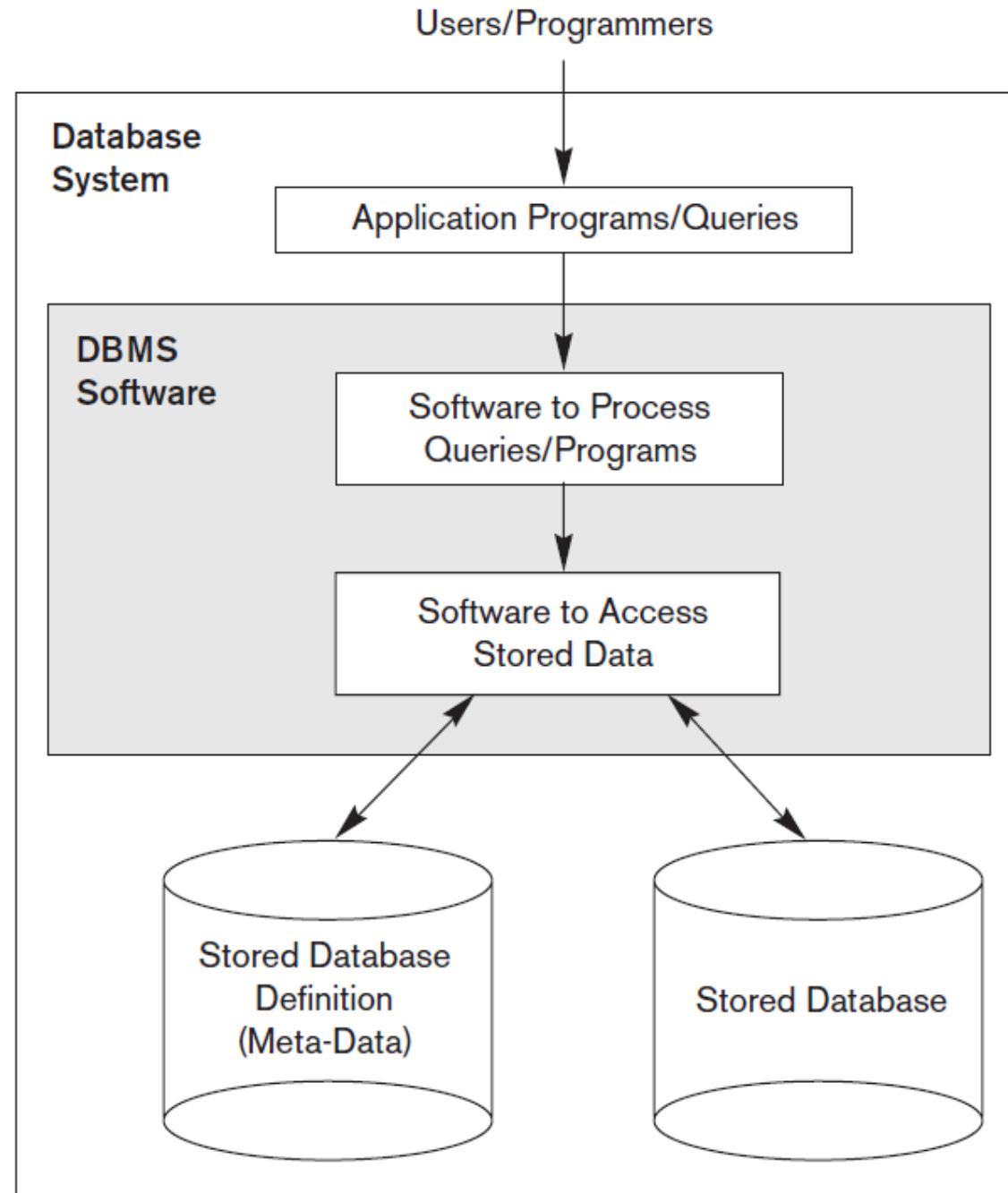
Contd...

Conceptual data model:-→ Entities, attributes, relationships

Relational Model:→Tables, rows, columns, primary/foreign keys

Physical model:→data types, storage mechanisms, indexing, and performance optimizations

Database System Environment



A simplified database system environment.

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

OVERVIEW OF DATABASE DESIGN:

The database design process can be divided into **six steps**. The **ER model** is most relevant to the **first three** steps:

(1) Requirements Analysis: what the users want from the database.

- understand what data is to be stored in the database
- what applications must be built on top of it, and
- what operations are most frequent and subject to performance requirements.

(2) Conceptual Database Design: Information gathered in the requirements analysis step is used to develop a high-level description of the data to be stored in the database, along with the constraints operating on the data.

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

(2) Conceptual Database Design: carried out using ER model.

(3) Logical Database Design: convert an ER schema into a relational database schema.

(4) Schema Refinement: To analyze the collection of relations in our relational database schema to identify potential problems, and to refine it (**normalizing relations**).

(5) Physical Database Design: involves building indexes on some tables, and clustering some tables, or a substantial redesign of parts of the database schema obtained from the earlier design steps.

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

(6) Security Design: Identify different user groups and different roles played by various users and accordingly restrict their access to the database system.

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

Entity: represents a real-world object or concept, that can be distinctly identified.

Entities are the fundamental building blocks of a database and are used to model data in an organized and meaningful way.

Examples: A person, an employee, car, product, student, department etc.

Attributes of an Entity: are the properties or characteristics of an entity.

- An entity is described using a set of attributes.
- All entities in a given entity set have the same attributes (similar).
- Examples: employee's name or salary, social security number (ssn), Phone

Number.

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

entity set: An entity set is a collection of similar entities that share the same attributes.

- entity sets need not be disjoint.
- Example: All employees in an organization form an "Employee" entity set.

Relationships Between Entities:

- A relationship among two or more entities represents an **association** among the entities.
- Example: a **works-on** relationship between an **employee** and a **project**.

Relationship set: a set of similar relationships.

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

Domain:

- refers to the set of all possible values that an attribute can take.

- Example: the domain associated with the attribute **name** of **Employees** might be the ***set of 20-character strings***.

For an attribute **Age**: Domain is : Integer values between 0 and 100.

For an attribute Gender: domain is : Male, Female, Non-Binary.

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

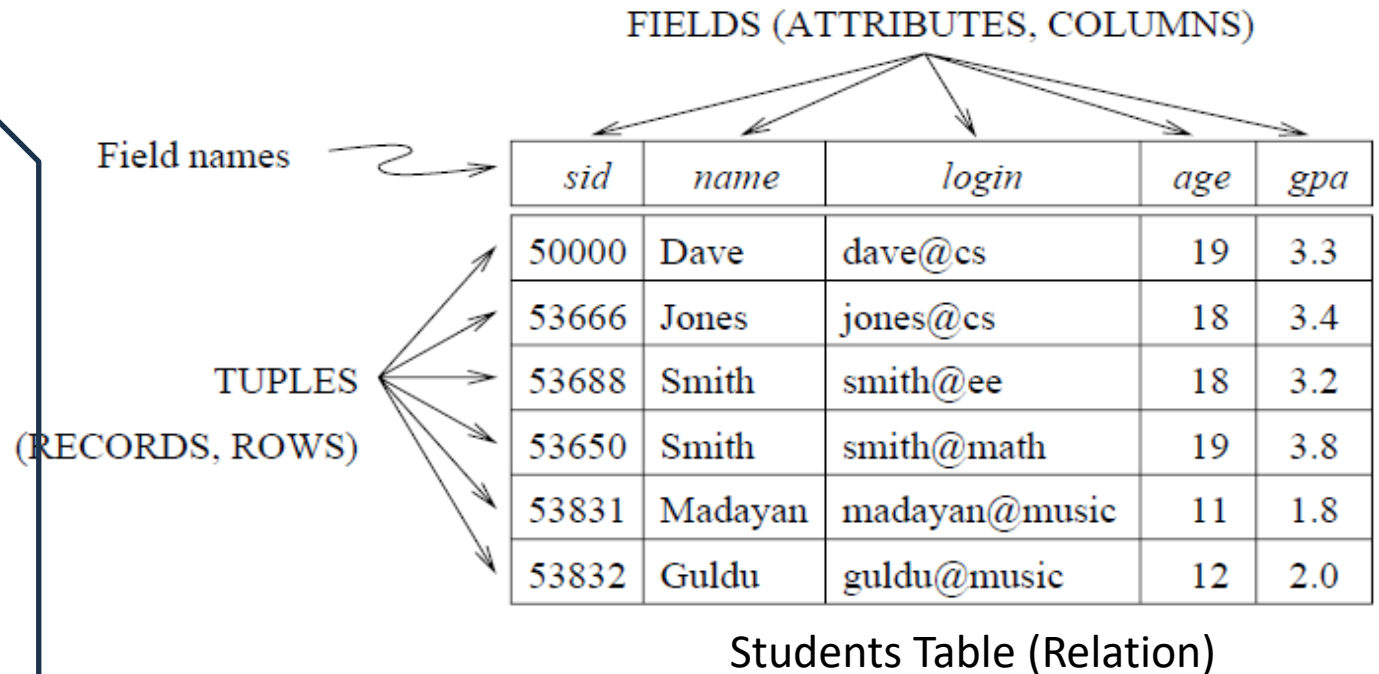
Relation: A **relation** is a **table** that represents data in a structured way, where the data is organized into **rows** and **columns**.

Tuples: A **tuple** is a single row in the relation.

- i.e. It represents a single row of data in a table, with each tuple containing values for each attribute.

There are 6 tuples and 5 fields.
No two rows are identical.

e.g.: Row representing
(50000, Dave, dave@cs, 19, 3.3)
is a **tuple** representing student
information.



Data Modeling Using the Entity–Relationship (ER) Model and Database Design

Key: is a minimal set of attributes whose values uniquely identify an entity in the set. There could be more than one **candidate key**; if so, we designate one of them as the **primary key**.

The **Employees** entity set with attributes **ssn**, **name**, and **lot** is shown in Figure below.

An **entity set** is represented by a **rectangle**
an **attribute** is represented by an **oval**.
the **primary key-ssn** is underlined.

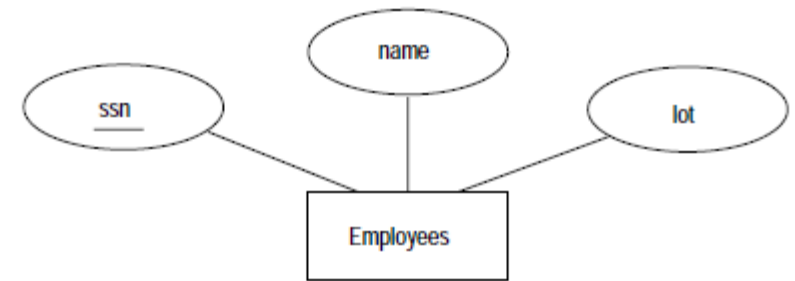


Figure : The Employees Entity Set

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

Relationships and Relationship sets:

A **relationship** is an association among two or more entities. A set of similar relationships form a **relationship set**.

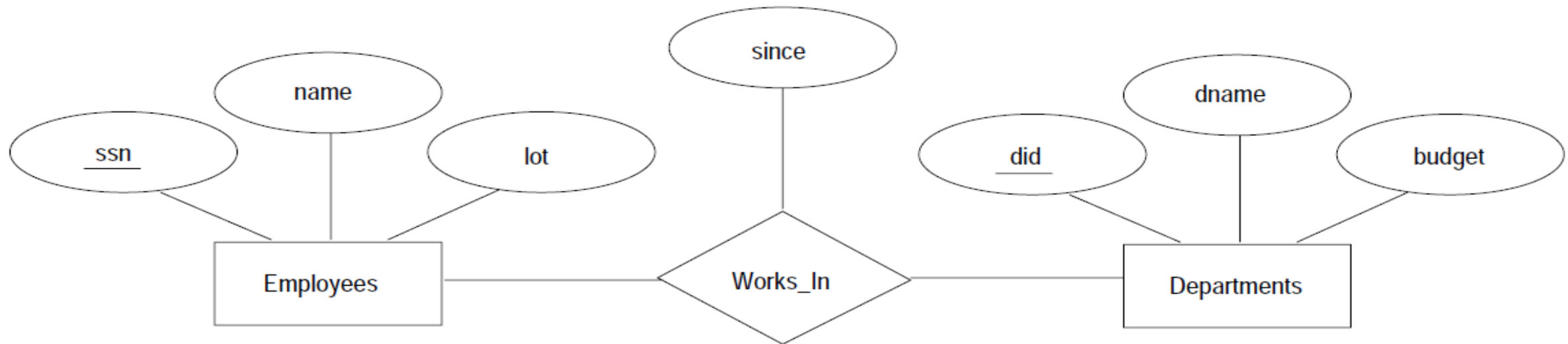


Figure 2.2 The Works_In Relationship Set

Descriptive attribute: are used to record information about the relationship, rather than about any one of the participating entities; e.g. : ***since***

There may be a **Manages** relationship between **Employees** and **Departments**

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

Schemas, Instances, and Database State:

Database Schema (intension):

- The description of a database is called the database schema.
- It represents the logical view of the entire database, detailing how the data is organized and how the relationships among different data elements are associated.
- Specified during database design.
- Not expected to change frequently.

Schema diagram:

- Most data models have certain conventions for displaying schemas as diagrams.

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

A **database** that stores student and course information.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson

schema diagram for the database

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

The diagram displays the structure of each record type but not the actual instances of records.

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

database state or snapshot (extension): refers to the **data** in the database at a *particular moment in time*.

It is also called the current set of ***occurrences*** or ***instances*** in the database.

*Difference between Database **schema** and Database **state**?*

empty state with no data.

database is first populated or loaded with the initial data-->**initial state**

Data Modeling Using the Entity–Relationship (ER) Model and Database Design

Valid state : A state that satisfies the structure and constraints specified in the schema.

DBMS Catalog: The **DBMS catalog** (or system catalog) contains **metadata** about the entire database system, and this includes details about the **schemas** within the database. i.e. The DBMS stores the descriptions of the schema constructs and constraints—also called the **meta-data**—in the **DBMS catalog**. The catalog keeps track of the structure of each schema.

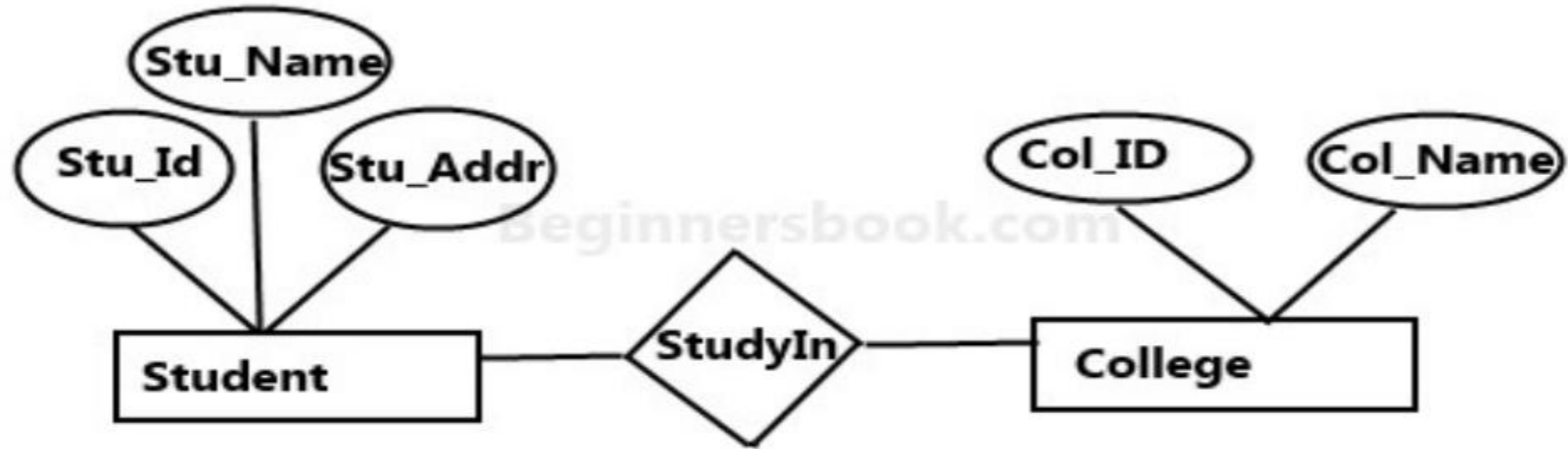
Entity–Relationship (ER) Diagram

- ER model stands for an Entity-Relationship model, which is a high-level data model.
- This model is used to identify the entities which exist in a system and the relationships between those entities.
- An ER model describes the structure of a database with the help of a diagram, which is known as **Entity Relationship Diagram (ER Diagram)**.
- An ER model is a design or blueprint of a database that can later be implemented as a database. The main components of E-R model are: **entity set** (has **attributes**) and **relationship set**.

Entity–Relationship (ER) Diagram

- It provides a preview of how all your tables should connect; what fields are going to be on each table.
- ER diagrams are translatable into relational tables which allows you to build databases quickly.
- An ER diagram has the following components:
 - **Rectangle:** Represents Entity sets.
 - **Ellipses:** Attributes
 - **Diamonds:** Relationship Set
 - **Lines:** They link attributes to Entity Sets and Entity sets to Relationship Set
 - **Double Ellipses:** Multivalued Attributes
 - **Dashed Ellipses:** Derived Attributes
 - **Double Rectangles:** Weak Entity Sets
 - **Double Lines:** Total participation of an entity in a relationship set

Entity–Relationship (ER) Diagram



Sample E-R Diagram

Entity–Relationship (ER) Model: ENTITY

- An entity is an object in the real world with an independent existence and can be differentiated from other objects.
- An entity might be
 - An object with physical existence, for e.g. a faculty or a student.
 - An object with conceptual existence, for e.g. a course, a job.

Classification of Entities

Entities are classified as strong entities and weak entities.

i) Strong Entities

An entity is considered a 'strong' entity, if it has the following properties:

- If it can exist apart from all of its related entities. i.e. A strong entity is not dependent of any other entity in the schema.
- It **always contains** a **Primary key** represented by the underline symbol.
- It is represented by a **rectangle** symbol.
- The relationship of two strong entities is represented by a **single diamond**.

Entity–Relationship (ER) Model: ENTITY

ii) Weak Entities

- An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called **weak entity**.
 - Unlike a strong entity, a **weak entity does not** have any *primary key*. It instead has a **partial discriminator key**. It can be identified uniquely by considering the *primary key of another entity*.
 - The weak entity is represented by a **double rectangle**.
- The relation between one strong and one weak entity is represented by a **double diamond**.
 - **example:** a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.

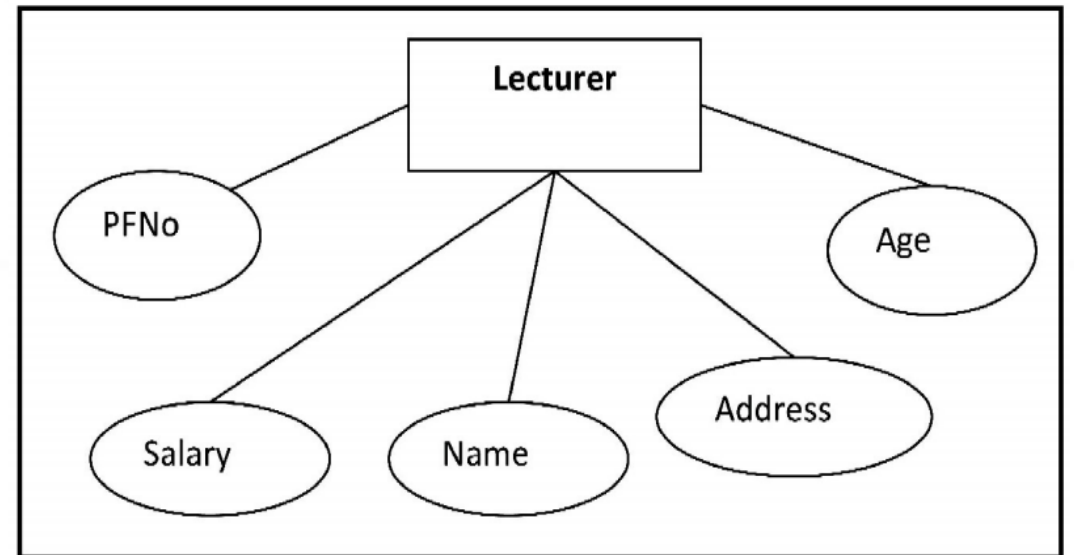


Entity–Relationship (ER) Model: ENTITY

e.g.: the relationship between Customer and Loan can be considered a weak relationship if the Loan entity depends on the existence of the Customer entity for its identification.

Entity–Relationship (ER) Model: ATTRIBUTES

- An attribute describes the property of an entity.
- An attribute is represented as Oval in an ER diagram.
- There are four types of attributes:
 - Key attribute
 - Simple attribute (single valued attribute)
 - Composite attribute
 - Multivalued attribute
 - Derived attribute

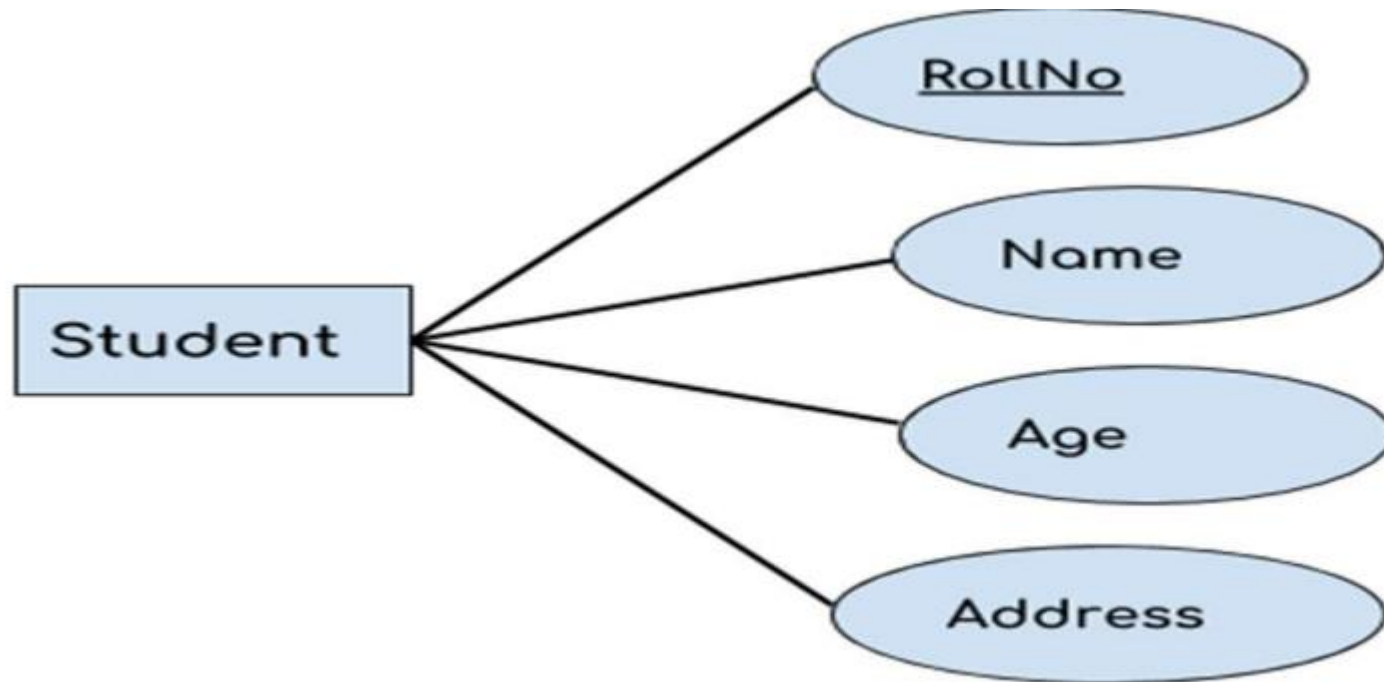


attributes of 'Lecturer' entity

Entity–Relationship (ER) Model: ATTRIBUTES

Key attribute

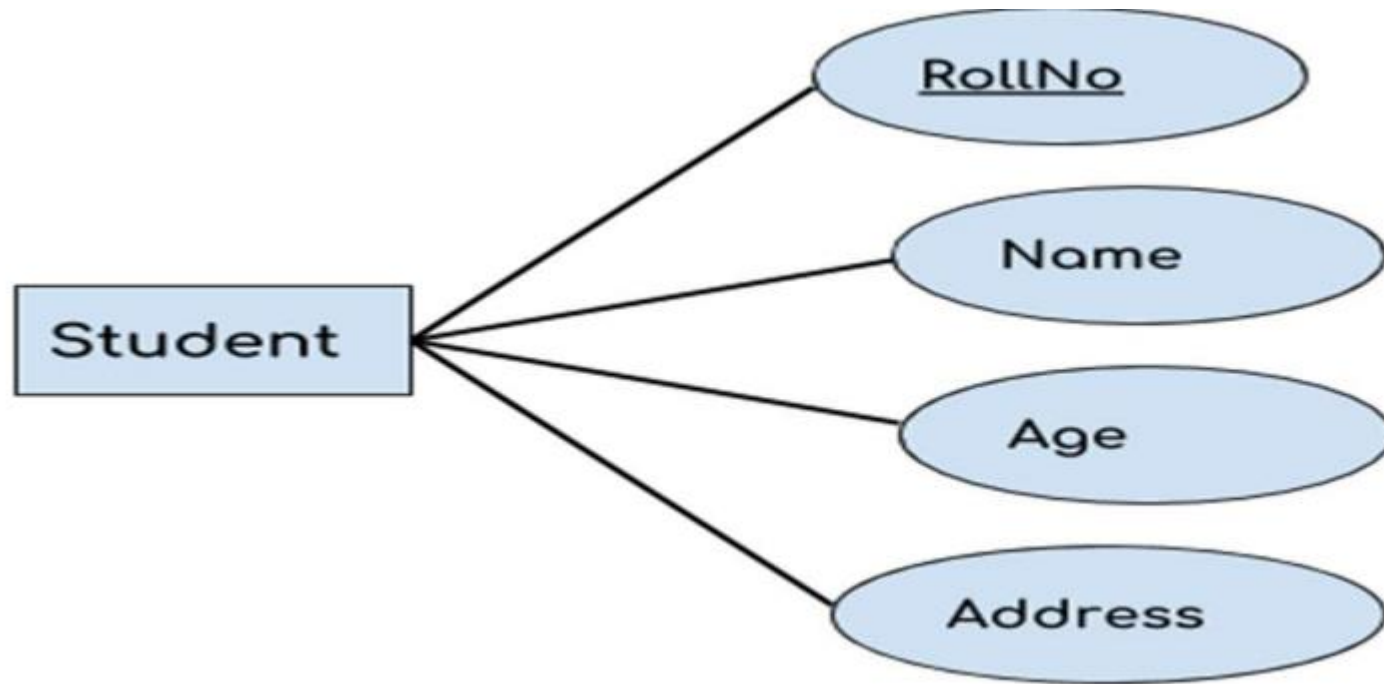
- Is an attribute that can uniquely identify an entity from an entity set.
 - For example, student **roll number** can uniquely identify a student from a set of students. Key attribute is represented by oval with an **underlined line**.



Entity–Relationship (ER) Model: ATTRIBUTES

Simple attribute

- Simple attributes are atomic values, which cannot be divided further. For example, In the diagram, **RollNo & Age** are the **Simple attributes**.

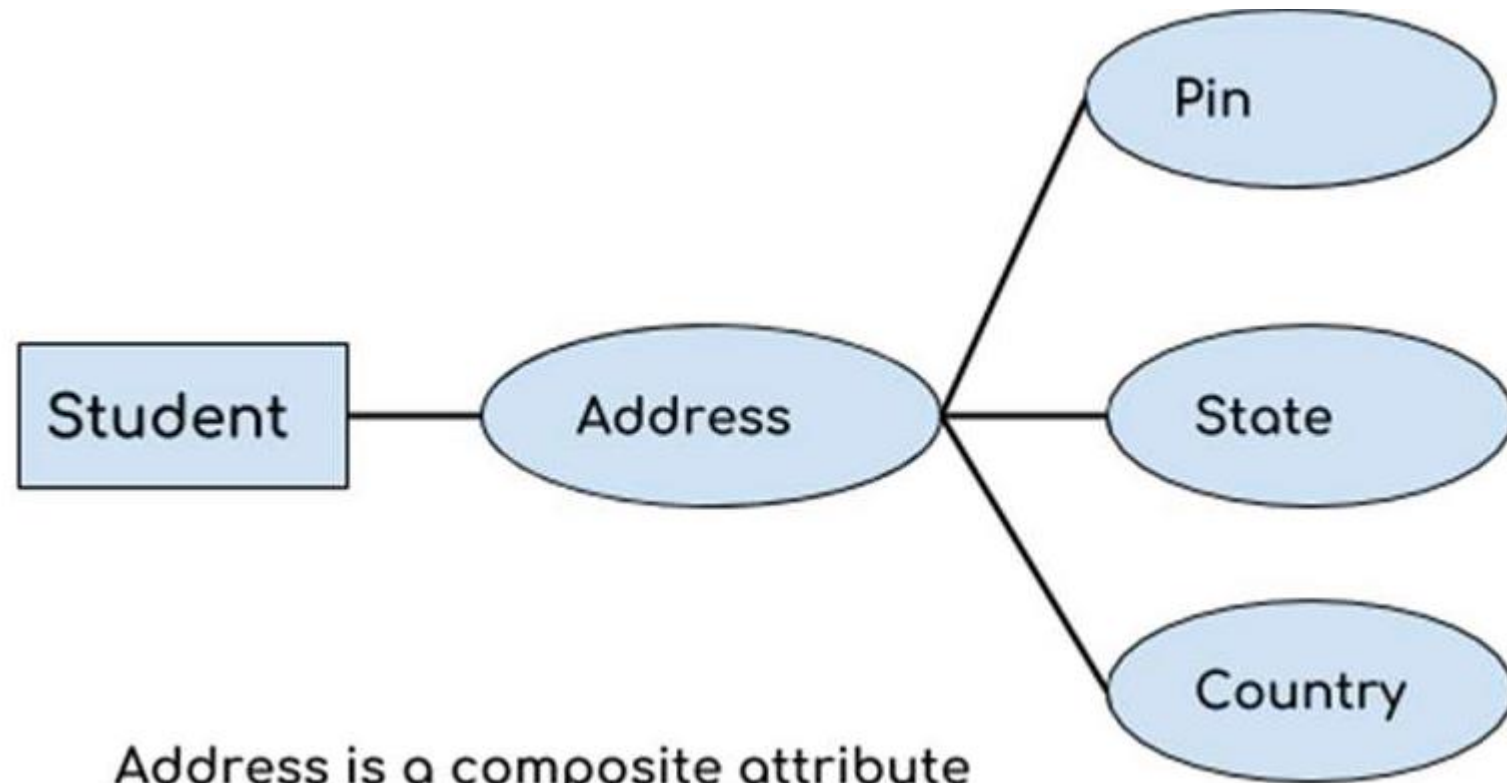


Entity–Relationship (ER) Model: ATTRIBUTES

Composite attribute

An attribute composed of many other attribute is called as composite attribute. For example, Address attribute of student Entity type consists of Street, City, State, and Country.

In ER diagram, composite attribute is represented by an oval comprising of ovals. For example, a student's complete name may have first_name and last_name.



Entity–Relationship (ER) Model: ATTRIBUTES

Multi valued attribute

Multi-value attributes may contain more than one values. For example, a person can have more than one phone number, email_address, etc.



Derived attribute

Derived attributes are the attributes that do not exist in the physical database, but their values are derived from other attributes present in the database.

For example, Age (can be derived from DOB). In ER diagram, derived attribute is represented by dashed oval.



Keys

- Key is an important constraint on an entity.
- The key is an attribute or a group of attributes whose values can be used to uniquely identify an individual entity in an entity set.
- Various types of keys are:
 - Candidate Key
 - Primary Key
 - Secondary key
 - Composite key
 - Alternate key
 - Foreign key

Keys

Candidate Key

- A candidate key is a simple or composite key that is **unique** and **minimal**.

- i.e. It is a set of one or more attributes (columns) that can uniquely identify a record (row) in a table.

e.gs: UNIVERSITY database example, if the entity is **Lecturer** (LID, FirstName, LastName, Address, Phone, BirthDate, Salary, DepartmentID), possible candidate keys are:

- LID**
- First Name and Last Name** - assuming there is no one else in the company with the same name.
- Last Name and DepartmentID** - assuming two people with the same last name don't work in the same department

Unique: no two rows in a table may have the same value for candidate key at any time.

Minimal: If a candidate key consists of multiple attributes, removing any attribute from the key would make it no longer unique. i.e. No unnecessary (redundant) attributes .

Keys

Primary Key

- The primary key is a *candidate key* that uniquely identifies each record (or row) in a table.
- The primary key must always have a value. It **cannot contain NULL** values, as a **NULL** would violate the **uniqueness** constraint and make it impossible to uniquely identify a record.

The primary key is indicated in the ER model by underlining the attribute.

e.g.: UNIVERSITY database example, if the entity is **Lecturer** (LID, FirstName, LastName, Address, Phone, BirthDate, Salary, DepartmentID), **primary key is LID**.

Keys

Composite key

- A composite key is composed of two or more attributes, that together can uniquely identify each row in the table, but it must be minimal.

e.gs: UNIVERSITY database example, possible composite keys are:

- First Name and Last Name - assuming there is no one else in the company with the same name.
- Last Name and Department ID - assuming two people with the same last name don't work in the same department.

Keys

Secondary key

- A secondary key in a database refers to an attribute (or a set of attributes) that is used to access data in a table but is not the primary key.
- It can be composite and used strictly for retrieval purposes.
- **egs:** in UNIVERSITY database example, Phone and Last Name are secondary keys.

Keys

Alternate key

- Alternate keys are all candidate keys not chosen as the primary key.
- An alternate key must have unique values (no duplicates) and cannot be NULL, just like the primary key.
- **egs:** In UNIVERSITY database example, if the entity is **Lecturer** (LID, FirstName, LastName, Address, Phone, BirthDate, Salary, DepartmentID), alternate keys are:
FirstName, LastName, Address, Phone, BirthDate, Salary, DepartmentID

Keys

Foreign key

- A foreign key (FK) is an attribute in a table that references the **primary key** in *another table* OR it can be null.
- i.e. It is an attribute (or a set of attributes) in one table that links to the **primary key** of *another table*.
- Both foreign and primary keys must be of the *same data type*.
eg: In the UNIVERSITY database, **DepartmentID** is the foreign key in the relation

Lecturer : Lecturer (LID, First Name, Last Name, Address, Phone, BirthDate, Salary, **DepartmentID**)

Department (**DepartmentID**, DepartmentName, DepartmentSpecialities)

Entity–Relationship (ER) Model: RELATIONSHIPS

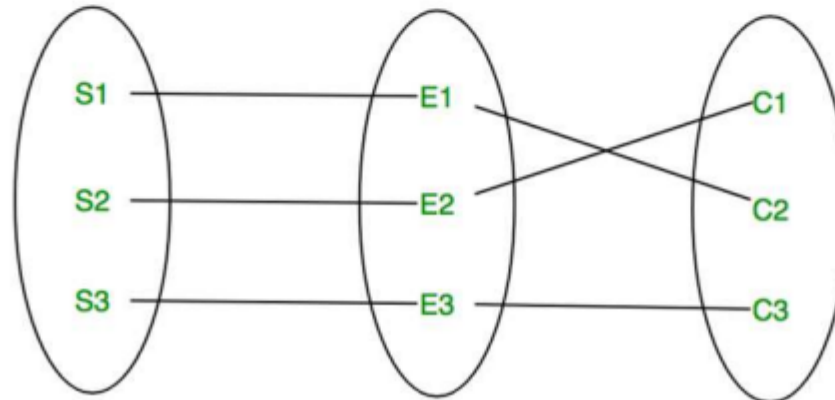
ER Model (Entity Relationship Model)

Relationship Type and Relationship Set

A relationship type represents the **association between entity types**. For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course. In ER diagram, relationship type is represented by a diamond and connecting the entities with lines.



A set of relationships of same type is known as relationship set. The following relationship set depicts S1 is enrolled in C2, S2 is enrolled in C1 and S3 is enrolled in C3.



Entity–Relationship (ER) Model: **RELATIONSHIPS**

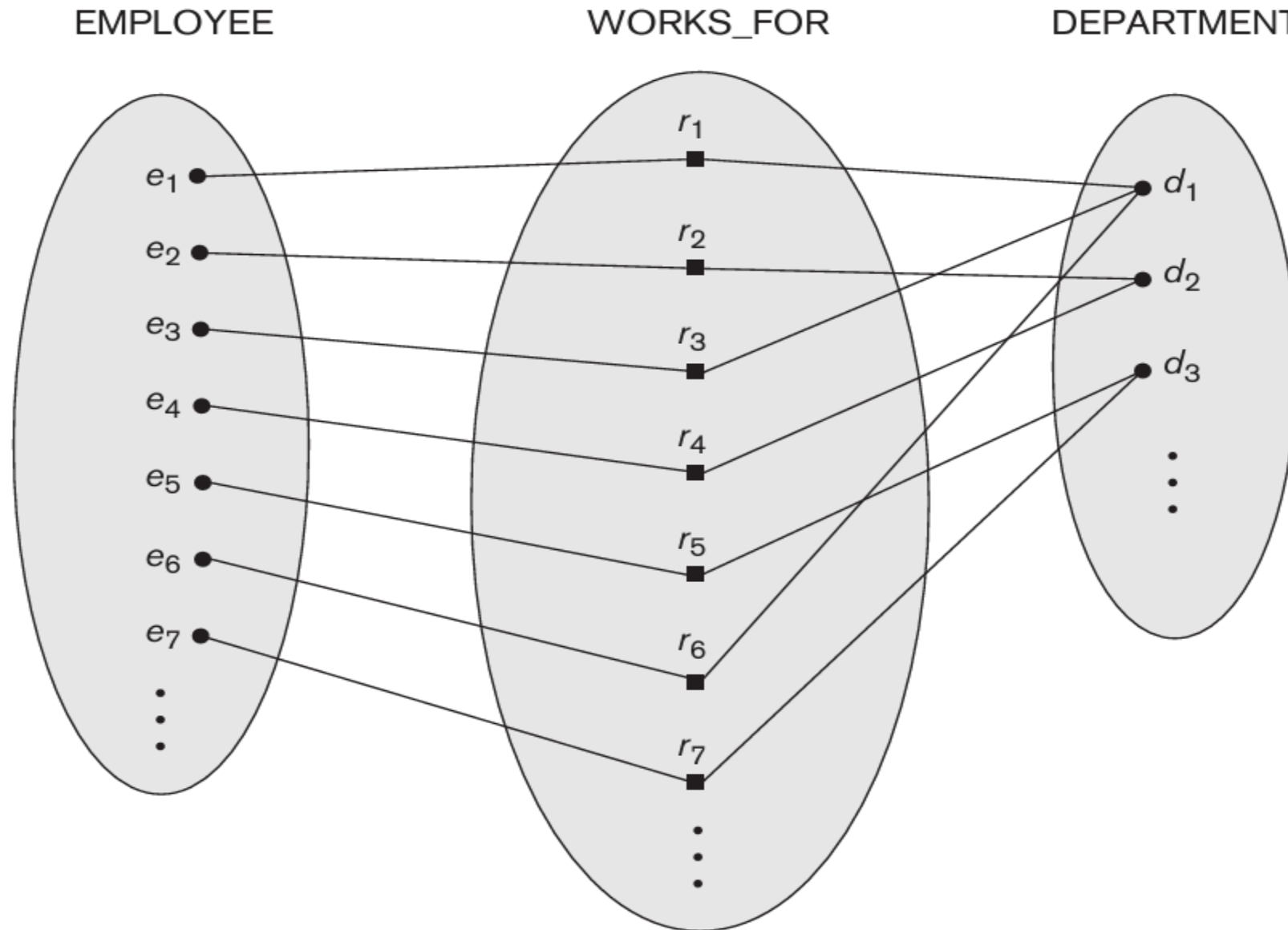


Figure 3.9

Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Entity–Relationship (ER) Model: **RELATIONSHIPS**

Degree of a relationship set

The number of different entity sets participating in a relationship set is called as degree of a relationship set.

Unary Relationship

When there is **only ONE** entity set participating in a relation, the relationship is called as unary relationship.

→ employee-manager
HoD-Faculty

Binary Relationship

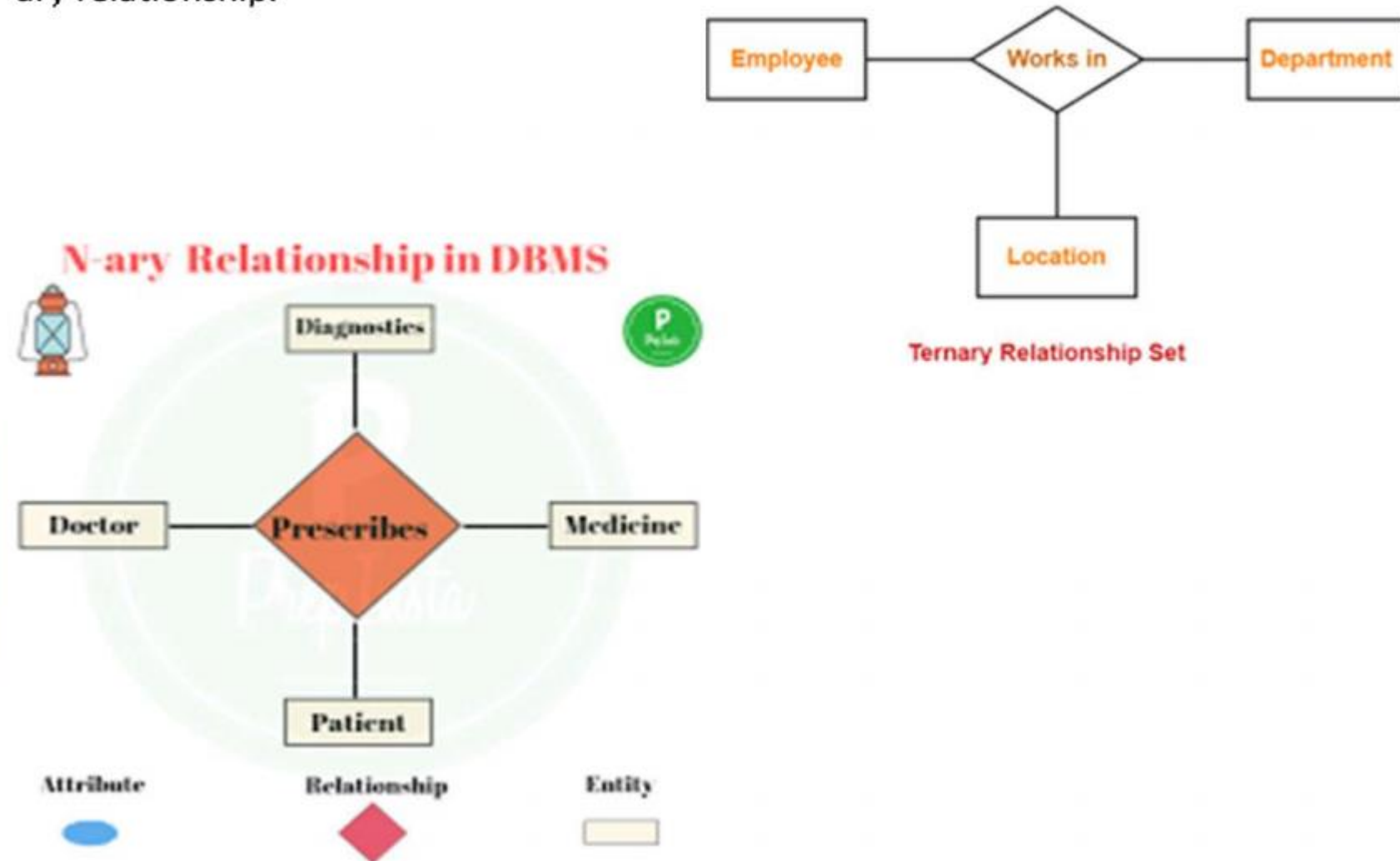
When there are **TWO** entities set participating in a relation, the relationship is called as binary relationship. For example, Student is enrolled in Course.



Entity–Relationship (ER) Model: **RELATIONSHIPS**

n-ary Relationship –

When there are n entities set participating in a relation, the relationship is called as n-ary relationship.



Entity–Relationship (ER) Model: **RELATIONSHIPS**

Cardinality of a Relationship:

The number of times an entity of an entity set participates in a relationship set is known as **cardinality**.

Cardinality can be of different types:

- One-to-one (1:1)
- One to many (1:N)
- Many-to-one (N:1)
- Many-to-many (M:N)

Entity–Relationship (ER) Model: **RELATIONSHIPS**

1. One to One Relationship (1:1):

When a single instance of an entity is associated with a single instance of another entity then it is called **one to one relationship**.

For example, a person has only one passport and a passport is given to one person.



Entity–Relationship (ER) Model: **RELATIONSHIPS**

One to Many Relationship (1:N):

When a single instance of an entity is associated with more than one instances of another entity then it is called one to many relationship.

For example – one department has many faculties, but each faculty can work in only one department.

Entity–Relationship (ER) Model: **RELATIONSHIPS**

Many to One Relationship (N:1):

When more than one instances of an entity is associated with a single instance of another entity then it is called many to one relationship.

For example – many students can study in a single college but a student cannot study in many colleges at the same time.

Entity–Relationship (ER) Model: **RELATIONSHIPS**

Many to Many Relationship (M:N):

When more than one instances of an entity is associated with more than one instances of another entity then it is called many to many relationship.

For example – an employee can be assigned to many projects and a project can have many employees.

Entity–Relationship (ER) Model: **RELATIONSHIPS**

Recursive Relationships (self-referencing relationships.):

Each entity type that participates in a relationship type plays a particular **role** in the relationship. The **role name** signifies the role that a participating entity from the entity type plays in each relationship instance, and it helps to explain what the relationship means.

For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of employee or worker and DEPARTMENT plays the role of department or employer.

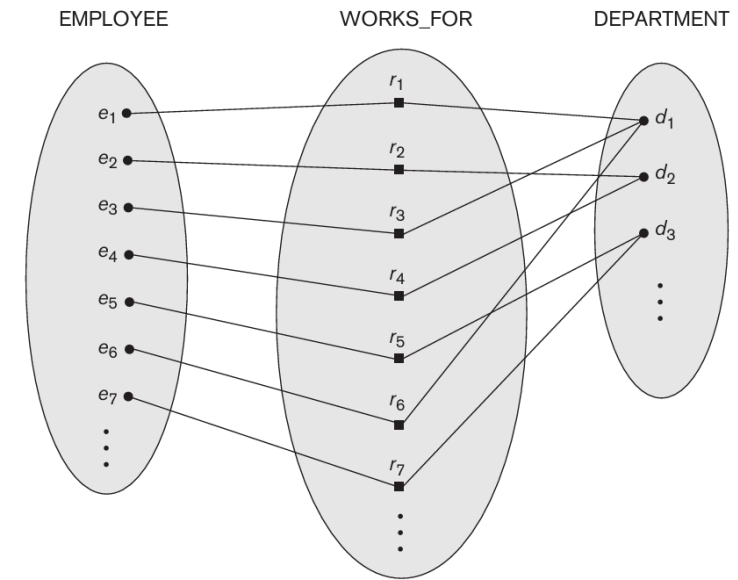


Figure 3.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Entity–Relationship (ER) Model: **RELATIONSHIPS**

In some cases the **same entity type** participates *more than once* in a relationship type in **different roles**. Such relationship types are called *recursive relationships* or *self-referencing relationships*.

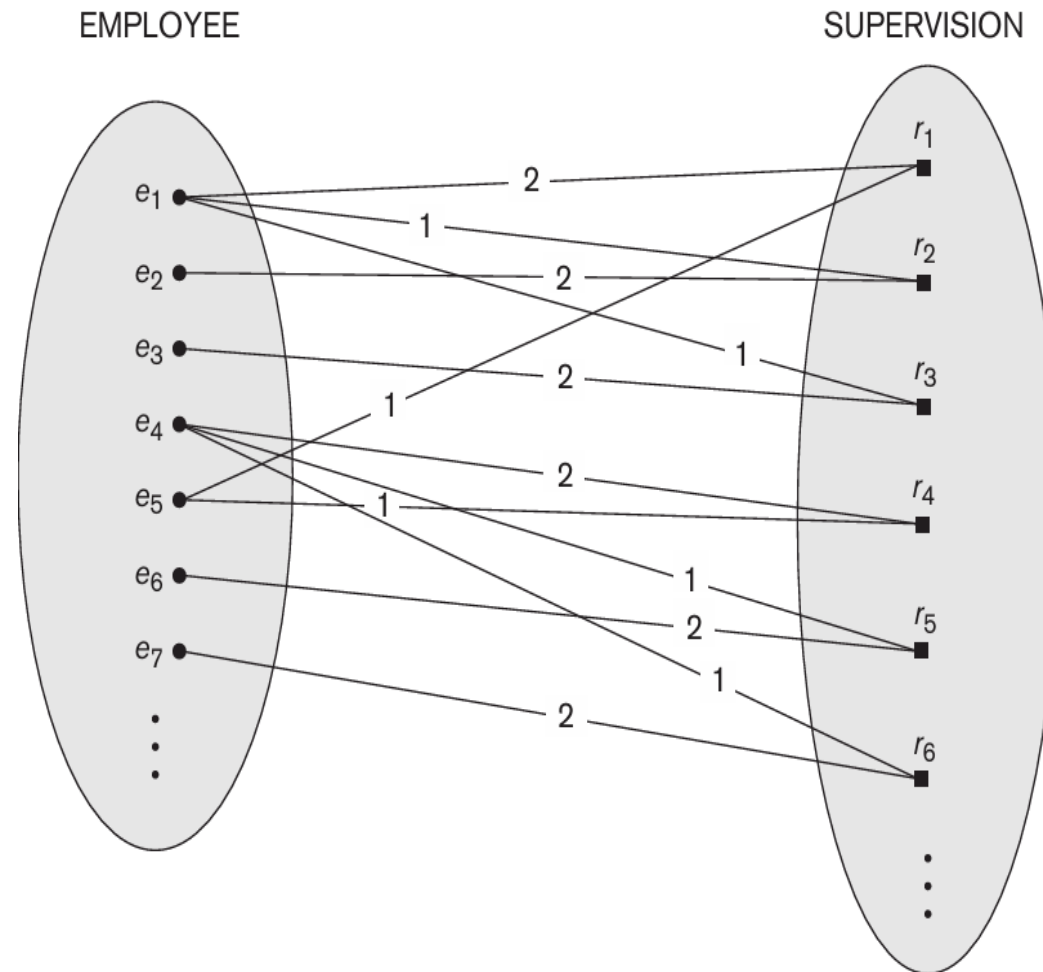


Figure 3.11

A recursive relationship
SUPERVISION
between EMPLOYEE
in the *supervisor* role
(1) and EMPLOYEE in
the *subordinate* role (2).

Entity–Relationship (ER) Model: **RELATIONSHIPS**

EMPLOYEE entity type participates **twice** in SUPERVISION: once in the role of ***supervisor (or boss)***, and once in the role of ***supervisee (or subordinate)***.

In Figure 3.11, the lines marked ‘1’ represent the supervisor role, and those marked ‘2’ represent the supervisee role; hence, e1 supervises e2 and e3, e4 supervises e6 and e7, and e5 supervises e1 and e4.

Participation Constraints and Existence Dependencies

- The **participation constraint** specifies whether the existence of an entity depends on its being related to another entity via the **relationship type**.
- This constraint specifies the *minimum number of relationship instances* that each entity can participate in and is some times called the **minimum cardinality constraint**.
- There are two types of participation constraints—**total and partial**.

Participation Constraints and Existence Dependencies

1. Total Participation of an Entity Set

- Each entity in the entity set **must participate** in the relationship.
- Total participation is also called **existence dependency**.
- Total participation is shown by **double line** in an ER diagram.
- e.g.1: If each student must enrol in a course, the participation of student will be total.
- e.g. 2: the participation of EMPLOYEE in WORKS_FOR is called total participation, meaning that every entity in the total set of employee entities must be related to a department entity via WORKS_FOR.

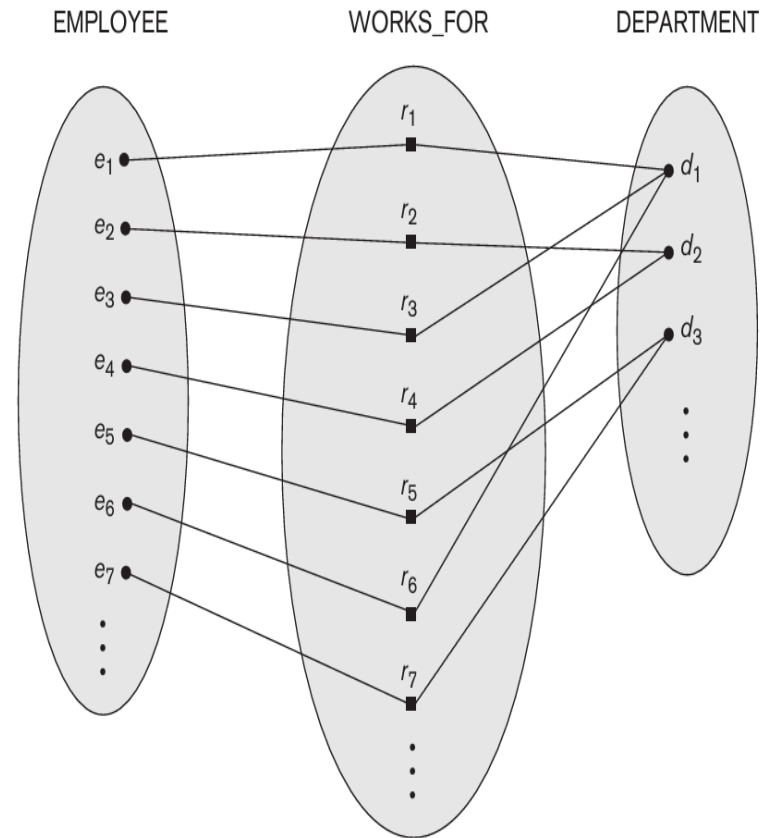


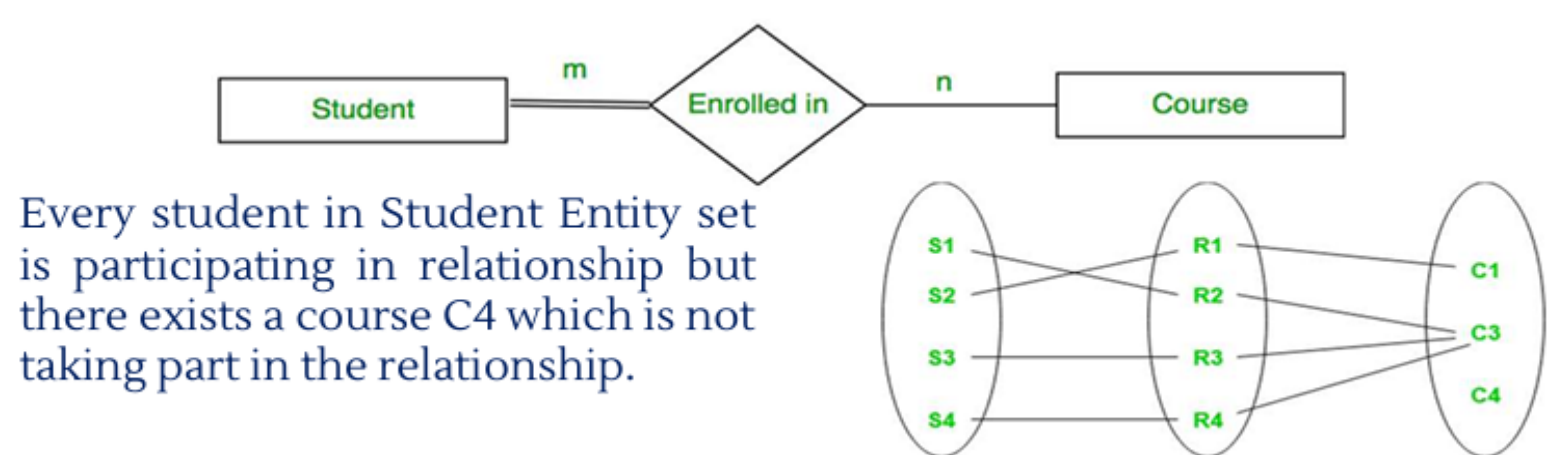
Figure 3.9

Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Participation Constraints and Existence Dependencies

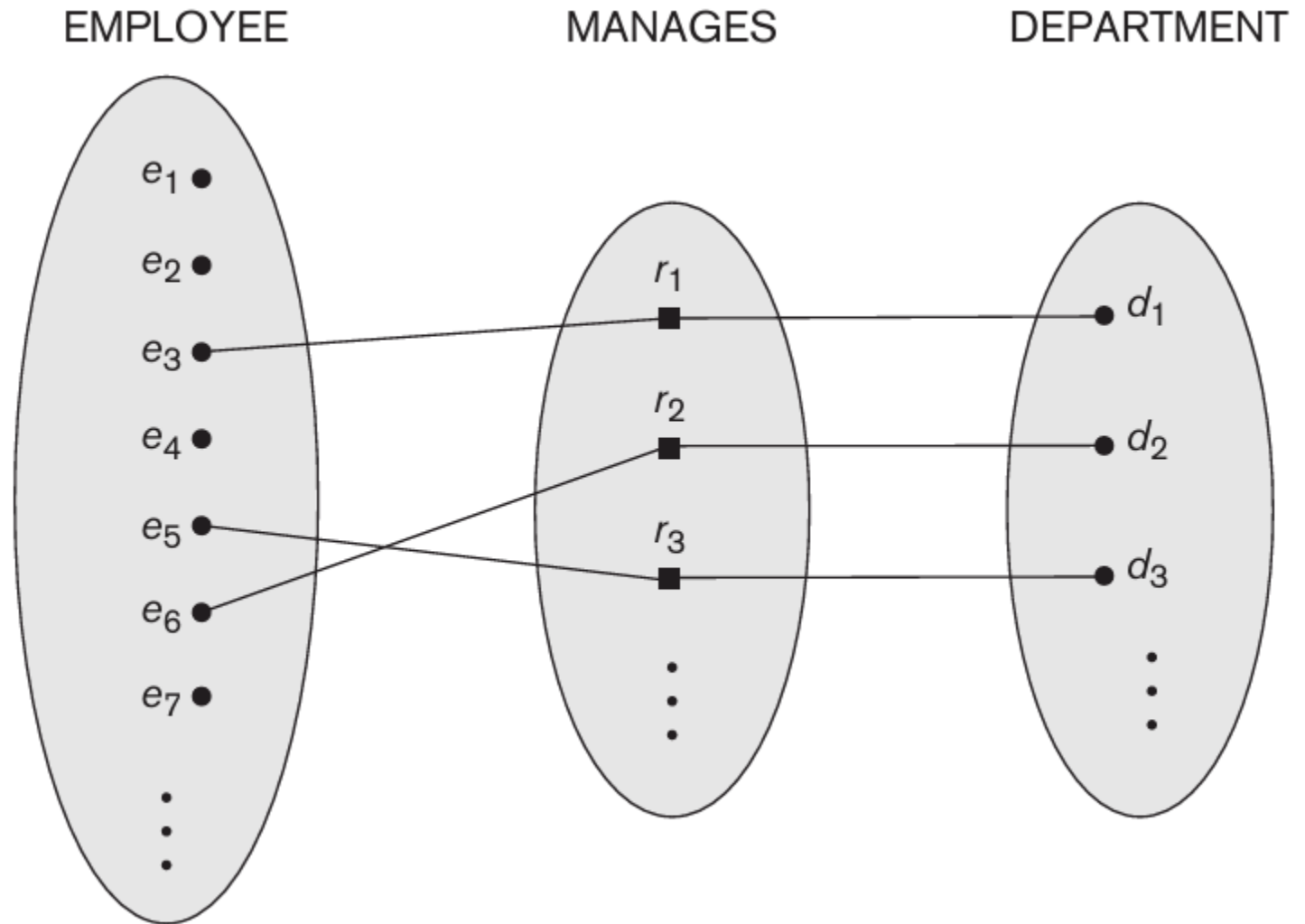
2. Partial Participation of an Entity Set

- The entity in the entity set **may or may NOT participate** in the relationship.
- i.e. ***some or part of*** the set of entities are related, but not necessarily all.
- partial participation is represented by a **single line** in an ER diagram.
- e.g.1: If some courses are not enrolled by any of the student, the participation of course will be partial.



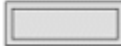
Participation Constraints and Existence Dependencies

e.g.2: Not every employee manages a department, so the participation of EMPLOYEE in the MANAGES relationship type is partial.



NOTATION for ER diagrams

Figure 3.14
Summary of the
notation for ER
diagrams.

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1: N for $E_1:E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

ER DIAGRAM- for a COMPANY Database

Create a database schema design based on the following (simplified) requirements of the COMPANY Database:

- The company is organized into DEPARTMENTS. Each department has a name, number and an employee who manages the department. We keep track of the start date of the department manager. A department may have several locations.
- Each department controls a number of PROJECTs. Each project has a unique name, unique number and is located at a single location.
- The database will store each EMPLOYEE's social security number, address, salary, gender, and birthdate. Each employee works for one department but may work on several projects.

ER DIAGRAM- for a COMPANY Database Contd...

- Each employee may have a number of DEPENDENTS. For each dependent, the DB keeps a record of name, gender, birthdate, and relationship to the employee.

Initial Conceptual Design of Entity Types for the COMPANY Database Schema

Based on the requirements, we can identify four initial entity types in the COMPANY database:

- DEPARTMENT
- PROJECT
- EMPLOYEE
- DEPENDENT

Initial Design of Entity Types: EMPLOYEE, DEPARTMENT, PROJECT, DEPENDENT

The initial attributes shown are derived from the requirements description.

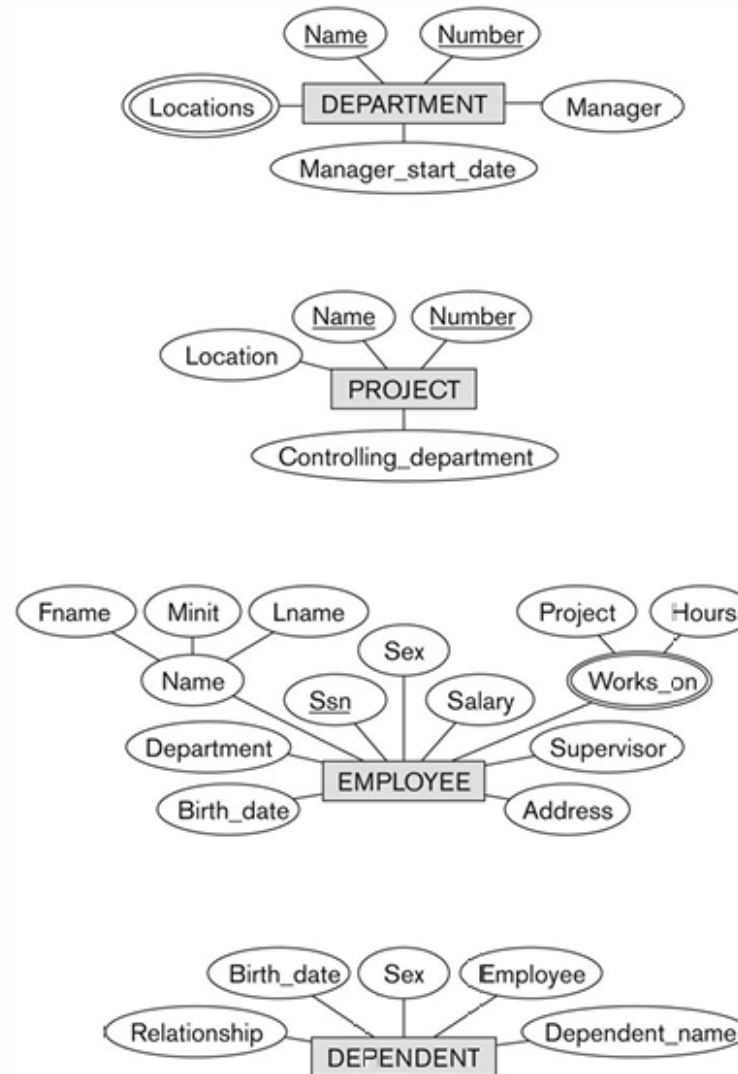


Figure 3.8
Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Refining the initial design by introducing Relationships

Some aspects in the requirements will be represented as **relationships**.

By examining the requirements, six relationship types are identified. The relationships with their participating entity types:

- **WORKS_FOR** (between EMPLOYEE, DEPARTMENT)
- **MANAGES** (also between EMPLOYEE, DEPARTMENT)
- **CONTROLS** (between DEPARTMENT, PROJECT)
- **WORKS_ON** (between EMPLOYEE, PROJECT)
- **SUPERVISION** (between EMPLOYEE (as subordinate), EMPLOYEE (as supervisor))
- **DEPENDENTS_OF** (between EMPLOYEE, DEPENDENT)

ER DIAGRAM- for a COMPANY Database Contd...

The cardinality ratio and participation constraint of each relationship type are determined from the requirements specification.

- **WORKS_FOR**, a 1:N (one-to-many) relationship type between DEPARTMENT and EMPLOYEE. Both participations are total.
- **MANAGES**, a 1:1(one-to-one) relationship type between EMPLOYEE and DEPARTMENT. EMPLOYEE participation is partial. DEPARTMENT participation is total.
- **CONTROLS**, a 1:N relationship type between DEPARTMENT and PROJECT. The participation of PROJECT is total, whereas that of DEPARTMENT is determined to be partial.

- **WORKS_ON**, determined to be an M:N (many-to-many) relationship type, with both participations are determined to be total.
- **SUPERVISION**, a 1:N relationship type between EMPLOYEE (in the supervisor role) and EMPLOYEE (in the supervisee role).
- **DEPENDENTS_OF**, a 1:N relationship type between EMPLOYEE and DEPENDENT, which is also the identifying relationship for the weak entity type DEPENDENT. The participation of EMPLOYEE is partial, whereas that of DEPENDENT is total.

ER DIAGRAM – Relationship Types are: WORKS_FOR, MANAGES, WORKS_ON, CONTROLS, SUPERVISION, DEPENDENTS_OF

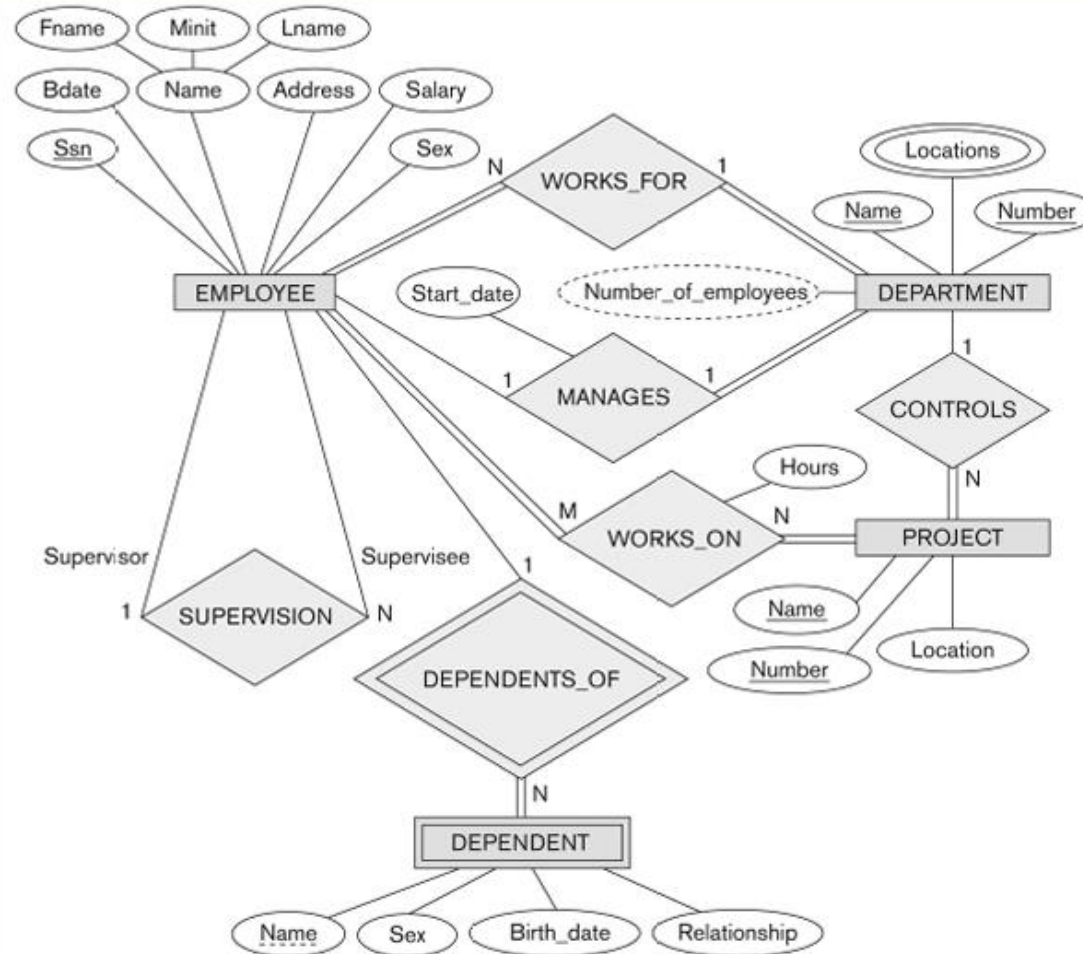


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Relationship Cardinality



One



Many



One and only one



Zero or one



One or many



Zero or many

e.g. 2: Music Database

The music database stores details of a personal music library, and could be used to manage your MP3, CD, or vinyl collection.

Since this database is for a personal collection, it's relatively simple and stores only the relationships between artists, albums, and tracks.

It ignores the requirements of many music genres, making it most useful for storing popular music and less useful for storing jazz or classical music.

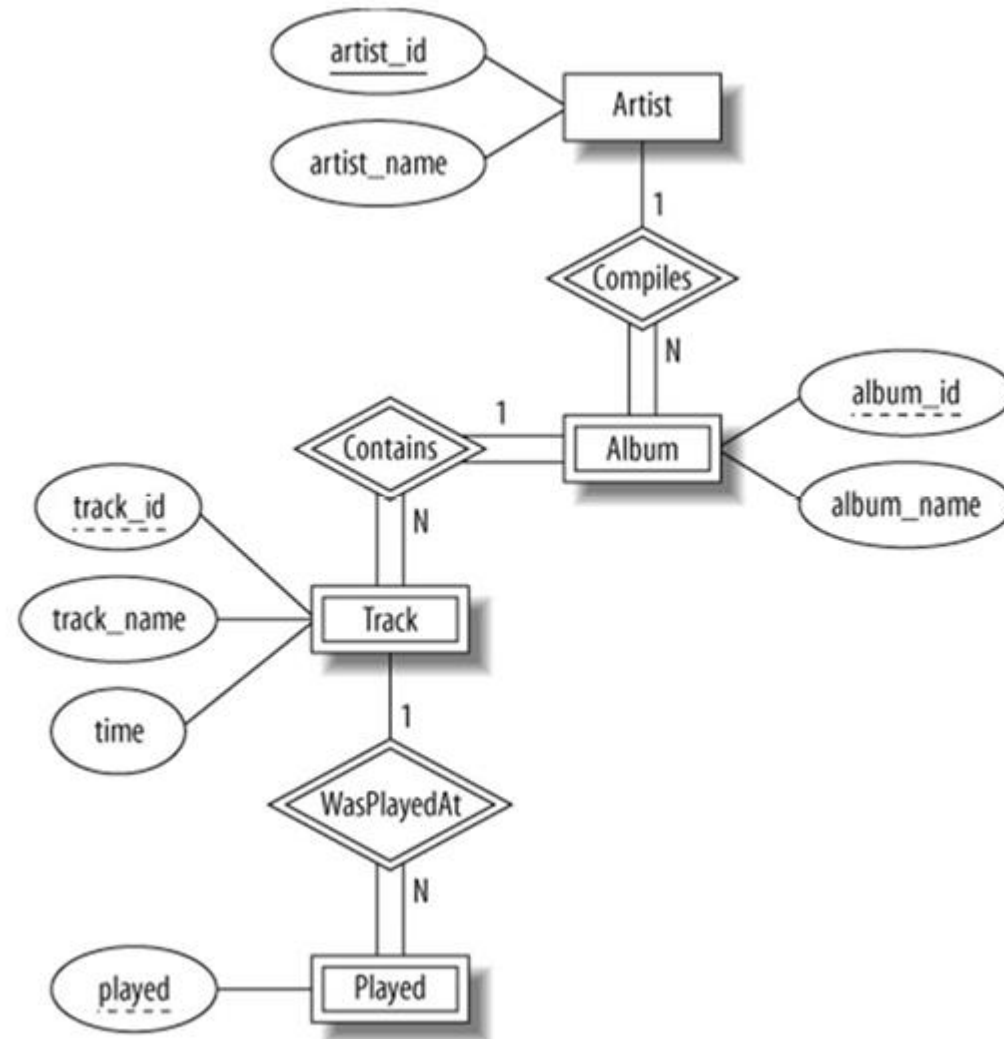
We first draw up a clear list of requirements for our database:

- The collection consists of albums.
- An album is made by exactly one artist.
- An artist makes one or more albums.
- An album contains one or more tracks
- Artists, albums, and tracks each have a name.
- Each track is on exactly one album.
- Each track has a time length, measured in seconds.
- When a track is played, the date and time the playback began (to the nearest second) should be recorded; this is used for reporting when a track was last played, as well as the number of times music by an artist, from an album, or a track has been played.

There's no requirement to capture composers, group members or sidemen, recording date or location, the source media, or any other details of artists, albums, or tracks.

e.g. 2: Music Database

Construct an ER diagram for the music database following up the given requirements.



e.g. 3: BANK MANAGEMENT SYSTEM

Construct an ER diagram for the BANK management system.

e.g. 4: PHARMACY DATABASE SYSTEM

The Prescriptions-R-X chain of **pharmacies** has offered to give you a free lifetime supply of medicines if you design its database. Given the rising cost of health care, you agree. Here's the information that you gather:

- Patients are identified by an SSN, and their names, addresses, and ages must be recorded.
- Doctors are identified by an SSN. For each doctor, the name, specialty, and years of experience must be recorded.
- Each pharmaceutical company is identified by name and has a phone number.
- For each drug, the trade name and formula must be recorded. Each drug is sold by a given pharmaceutical company, and the trade name identifies a drug uniquely from among the products of that company. If a pharmaceutical company is deleted, you need not keep track of its products any longer.
- Each pharmacy has a name, address, and phone number.
- Every patient has a primary physician. Every doctor has at least one patient.
- Each pharmacy sells several drugs and has a price for each. A drug could be sold at several pharmacies, and the price could vary from one pharmacy to another.
- Doctors prescribe drugs for patients. A doctor could prescribe one or more drugs for several patients, and a patient could obtain prescriptions from several doctors. Each prescription has a date and a quantity associated with it. You can assume that if a doctor prescribes the same drug for the same patient more than once, only the last such prescription needs to be stored.

e.g. 4: PHARMACY DATABASE SYSTEM

- Pharmaceutical companies have long-term contracts with pharmacies. A pharmaceutical company can contract with several pharmacies, and a pharmacy can contract with several pharmaceutical companies. For each contract, you have to store a start date, an end date, and the text of the contract.
- Pharmacies appoint a supervisor for each contract. There must always be a supervisor for each contract, but the contract supervisor can change over the lifetime of the contract.

Draw an ER diagram that captures the above information. Identify any constraints that are not captured by the ER diagram.



MYSQL LAB INSTRUCTIONS

to install mysql:

```
sudo apt-get update
```

```
sudo apt-get install mysql-server
```

itdmin@itadmin-optiplex-9020:~\$ **sudo mysql**
Welcome to the MySQL monitor. Commands
end with ; or \g. Your MySQL connection id is 4

mysql>

If there's any issues in login , follow these instructions:

To uninstall mysql

```
sudo apt-get remove --purge mysql*
```

```
sudo apt-get autoremove
```

```
sudo apt-get autoclean
```

Thank You